SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1025	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	*Build full-stack applications with an emphasis on containerization, version control, and collaborative development*		
Problem Statement:	**Bus Ticket Reservation System (project: BusBuddy)**		
Student Name:	K. Sampath Reddy	**Reg. No.:**	192424285
Date:	20-10-2026	**Duration:**	60 minutes

Code of Conduct

I K Sampath Reddy (192424285) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __*K Sampath Reddy*____

Annexure A – Git & Docker Technical Assignment

BusBuddy: Version Control and Containerization

Technical report with Git repository, Dockerfile, Docker Compose stack and test evidence

No.	Section (Annexure A)	No.	Section (Annexure A)
1	Problem Analysis	7	Docker Environment Design
2	Project Structure Design	8	Dockerfile Development
3	Git Repository Initialization	9	Docker Compose Design
4	Git Branching Strategy	10	Container Deployment and Testing
5	Version Control Workflow	11	Benefits of Git and Docker
6	Commit History Analysis	12	Challenges and Improvements

1. Problem Analysis

1.1 The Problem Statement

The assigned problem statement is the **Bus Ticket Reservation System**. Passengers need to search buses, see which seats are really free, book a seat and cancel it for a refund, while operators need to manage trips. The difficult part is that many people try to buy the **same seat at the same time**. This report covers the project I created for this problem, **BusBuddy**, and shows how Git (version control) and Docker (containers) are used to build, share and run it.

BusBuddy has these unique features, which also decide how the code is organised: an **8-minute seat lock** (a seat is held while the passenger pays), **double-booking protection**, a **live seat map**, and a **slab-based refund** (the earlier you cancel, the more you get back).

1.2 Project Objectives

ID	Objective	How it is measured
O-1	Never sell one seat twice	Second booking of a locked or booked seat is rejected (HTTP 409 / 423)
O-2	Hold a seat for a short, fixed time	Redis lock expires after 480 seconds
O-3	Refund automatically by slab	90% / 50% / 25% / 0% by hours before departure
O-4	Keep every change traceable	Each change is one commit with a clear message
O-5	Run the same way on every machine	One command: docker compose up --build

1.3 Functional Requirements Used in This Assignment

The full requirements are in the earlier SRS (CO2). These were built into the repository for this assignment, together with a web page (frontend/) showing the seat map.

ID	Requirement	Where it is built	API call
FR-A	Search trips by source and destination	backend/src/server.js	GET /api/trips
FR-B	Show live seat status (available, held, booked)	server.js, locks.js	GET /api/trips/:id/seats
FR-C	Hold seats for 8 minutes	backend/src/locks.js	POST /api/trips/:id/lock
FR-D	Confirm a booking only for a locked seat	server.js, db.js	POST /api/bookings
FR-E	Cancel and calculate the refund	backend/src/refund.js	DELETE /api/bookings/:id

1.4 Expected Outcomes

- A Git repository with a clean, readable history (34 commits, 10 merges, 2 release tags).
- A working API with automated tests (6 tests, all passing), plus a Dockerfile and a Docker Compose file that start the whole system with one command.

- Diagrams, commands and screenshots that explain and prove each step.

2. Project Structure Design

The repository separates the three parts of the system (API, web page and proxy) so each can be changed and containerized on its own. Figure 1 shows the real folder tree produced from the repository.



Figure 1: *Folder structure of the BusBuddy repository (output of the tree command).*

2.1 Purpose of Each Major Folder and File

Folder / file	Purpose and justification
backend/	Node.js REST API. Kept in its own folder so it has its own dependencies, tests and Dockerfile.
backend/src/server.js	Creates the Express app and defines all routes (search, seats, lock, book, cancel).

Folder / file	Purpose and justification
backend/src/locks.js	Redis seat-lock logic (8-minute expiry). Separate file so the rule is easy to test.
backend/src/refund.js	Refund slab calculation. A small pure function, easy to unit test and to change.
backend/src/db.js	SQLite storage for trips and bookings; the UNIQUE seat rule prevents double booking.
backend/test/	Automated tests; fakeRedis.js lets tests run without a Redis server.
backend/Dockerfile	Recipe that builds the API image. Placed beside the code it packages.
backend/.dockerignore	Keeps node_modules, tests and secrets out of the image.
frontend/	Static web page (HTML, CSS, JavaScript) showing the seat map.
nginx/default.conf	Serves the web page and forwards /api calls to the API container.
docker-compose.yml	Starts api, redis and web together with network and volume.
.env.example	Template for settings (such as the web port). The real .env is not committed.
.gitignore	Tells Git which files to skip (node_modules, .env, *.db, logs).
.github/workflows/ci.yml	Continuous-integration pipeline: install, test, build images.
docs/GIT_WORKFLOW.md	Written branching and commit rules for the team.
README.md, CHANGELOG.md	How to run the project; what changed in each version.

3. Git Repository Initialization

3.1 Steps

The repository was created with the commands below. Each step has one purpose, explained in the table that follows.

```
$ mkdir busbuddy && cd busbuddy
$ git init -b main                      # create repository, first branch is "main"
$ git config user.name  "K. Sampath Reddy"
$ git config user.email "sampathkondu1@gmail.com"
$ git add .gitignore README.md          # stage the first files
$ git commit -m "chore: initialise repository with README and .gitignore"
$ git checkout -b develop               # integration branch
$ git remote add origin <path-or-url>   # link to the shared remote
$ git push -u origin main develop       # publish both branches
```

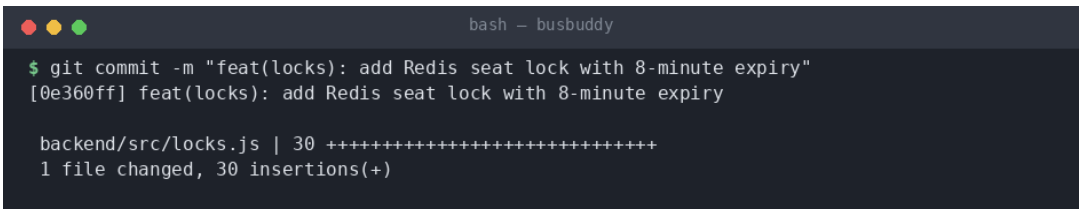


Figure 2: *The first real feature commit as recorded by Git (hash, message, files changed).*

3.2 Essential Git Files

File / folder	Purpose
.git/	Hidden folder where Git stores all history, branches and tags. Created by git init; never edited by hand.
.gitignore	Lists files that must never be committed: node_modules/ (large, re-creatable), .env (secrets), *.db (local data), *.log.
README.md	First page a new team member reads: what BusBuddy is, how to run it, how to test it.
package-lock.json	Locks exact dependency versions so every machine and every Docker build installs the same packages.
.env.example	Shows which settings exist without storing real secrets.
CHANGELOG.md	Human-readable list of changes per version (v1.0.0, v1.0.1).

4. Git Branching Strategy

BusBuddy uses a **Git Flow** style model with five kinds of branches. Figure 3 shows how they were actually used in this project: four feature branch groups, one release (v1.0.0) and one hotfix (v1.0.1).

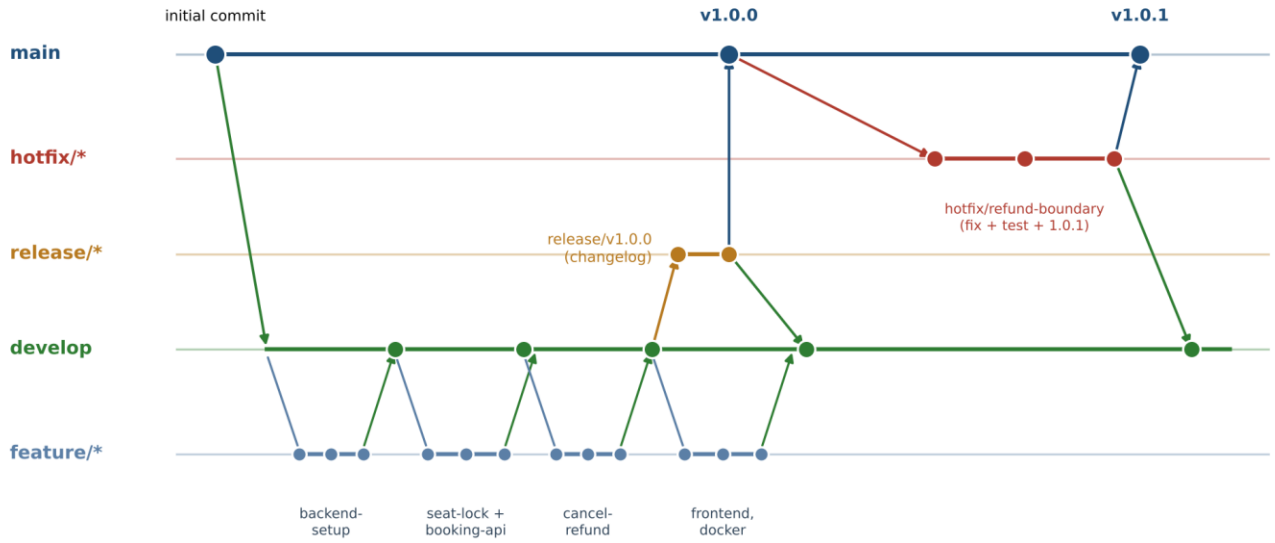


Figure 3: *Branching strategy used for BusBuddy. Arrows show where work is merged.*

Branch	Created from → merged into	Purpose in BusBuddy
main	– (long-lived)	Always stable. Only release and hotfix merges; each is tagged (v1.0.0, v1.0.1).
develop	main → main (via release)	Integration branch where finished features meet and are tested together.
feature/*	develop → develop	One branch per feature: seat-lock, booking-api, cancel-refund, frontend, docker.
release/*	develop → main and develop	Final checks, changelog and version number before a release.
hotfix/*	main → main and develop	Urgent fix to released code (refund boundary bug) without waiting for new features.

4.1 Why This Model Suits the Project

- **Money and trust are involved.** A bug in booking or refunds must be fixed fast on the released version. A hotfix branch does this without shipping half-finished features.
- **Features depend on each other** (booking needs seat locks), so a shared develop branch lets the team test them together before release.
- **Releases are planned** (bus schedules and fares change on dates), which matches release branches and version tags.
- **Clear naming** (feature/seat-lock, hotfix/refund-boundary) tells everyone what a branch is for.

5. Version Control Workflow

The workflow in Figure 4 was followed for every feature. The table lists each Git operation that was performed on the repository and why.

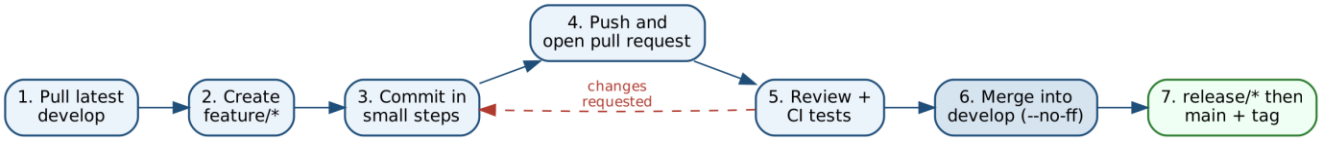


Figure 4: *Daily Git workflow followed for each BusBuddy feature.*

Operation (command)	Purpose
git checkout -b feature/seat-lock develop	Start new work on an isolated branch so develop stays stable.
git add <file> / git commit -m "..."	Record one logical change with a message that explains it.
git merge --no-ff feature/seat-lock	Bring a finished feature into develop and keep a visible merge commit.
git branch -d feature/seat-lock	Delete a merged branch to keep the repository tidy.
git push -u origin develop	Share commits with the remote; -u remembers the upstream branch.
git fetch origin / git pull	Download teammates' work; pull = fetch + merge into the current branch.

Operation (command)	Purpose
git tag -a v1.0.0 -m "..."	Mark a release permanently so it can be rebuilt or rolled back.
git log --oneline --graph --all	Inspect history and branch structure.

5.1 Merge Conflict and Its Resolution

Two branches, **feature/seat-lock** and **feature/booking-api**, both added a new route at the same place in backend/src/server.js and both changed the same function line. When the second was merged into develop, Git could not combine them and stopped with a conflict (Figure 5).



Figure 5: *Real merge conflict reported by Git when merging feature/booking-api.*

Resolution steps: (1) open the file and read both versions between the <<<<<<<, ======= and >>>>>>> markers; (2) keep both routes; (3) improve the booking route so it checks the seat lock (a decision that only became clear once both pieces were side by side); (4) run node --check to confirm the file is valid; (5) git add the file and git commit to finish the merge. The merge commit message records what was decided.

5.2 Repository Synchronization

A second clone acted as a teammate and pushed a commit to develop. My copy was behind by one commit; git fetch showed it and git pull brought it in as a fast-forward (Figure 6).



Figure 6: *Synchronizing with the shared remote: fetch, status and pull.*

6. Commit History Analysis

Figure 7 is the complete history of the repository. It contains **34 commits** (24 normal commits and 10 merge commits), **2 tags** and the branches described in Section 4. The left column continues in the right column.

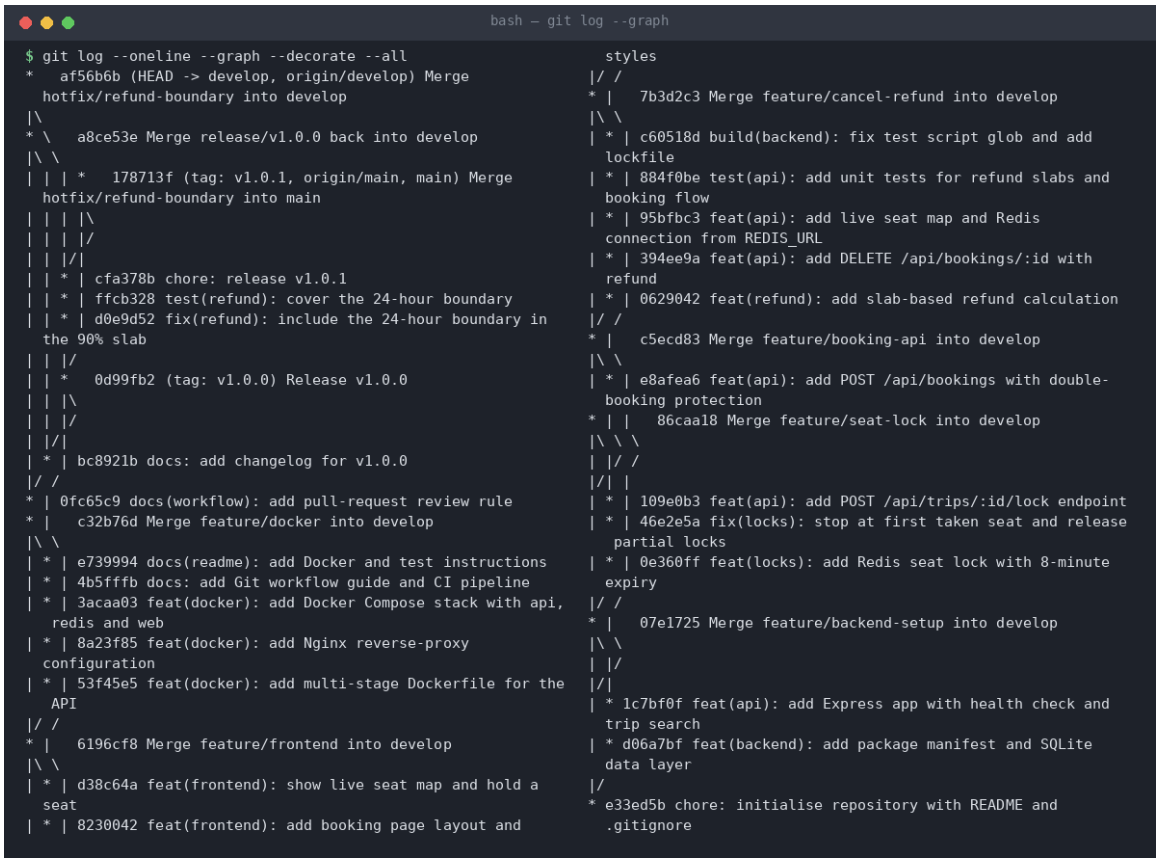


Figure 7: *Complete commit history (git log --oneline --graph --decorate --all).*

6.1 How the Commit Messages Follow Good Practice

Practice	Example from this repository	Why it helps maintenance
Type and scope prefix (Conventional Commits)	feat(locks): add Redis seat lock with 8-minute expiry	Readers can filter history by feature, fix, test, docs or build.
Short, present-tense summary	fix(refund): include the 24-hour boundary in the 90% slab	The line alone says what changed; easy to scan in git log.
One logical change per commit	test(refund): cover the 24-hour boundary	A bad change can be reverted without losing other work.
Explain decisions in the body	Merge commit: "Resolved conflict in server.js … made /api/bookings require a valid seat lock"	Future developers learn why, not only what.
Fix, then test, then release	fix → test → chore: release v1.0.1	Shows the fix is proved and versioned.
No generic messages	No "update", "changes" or "final"	Search and blame stay useful.

7. Docker Environment Design

7.1 Why Docker Suits BusBuddy

- **Several parts must run together:** API, Redis and web server. Compose starts them all with one command.
- **Same environment everywhere:** the API needs Node.js 22 and a running Redis. A container carries the exact versions, so "it works on my machine" problems disappear.
- **Easy scaling on busy days:** festival weekends bring many bookings; more API containers can be started from the same image.
- **Safe and clean:** the API runs as a non-root user, and nothing is installed on the host computer.

7.2 Components Containerized

Component	Image	Containerized?	Reason
api	busbuddy-api:1.0.0 (built from backend/Dockerfile)	Yes	Our own code; needs a fixed Node.js version and a data folder.
redis	redis:7-alpine (official)	Yes	Seat locks need Redis; no installation on the host.
web	nginx:1.27-alpine (official)	Yes	Serves frontend/ and forwards /api to the API.
SQLite file	Docker volume busbuddy-data	Stored outside the container	Bookings must survive container restarts.
Browser, Git	–	No	Used by the person, not part of the running system.

8. Dockerfile Development

The API image is built with a **multi-stage** Dockerfile (Figure 8). Stage 1 installs only production packages; Stage 2 copies the result into a small clean image. This keeps the final image small and free of build leftovers.

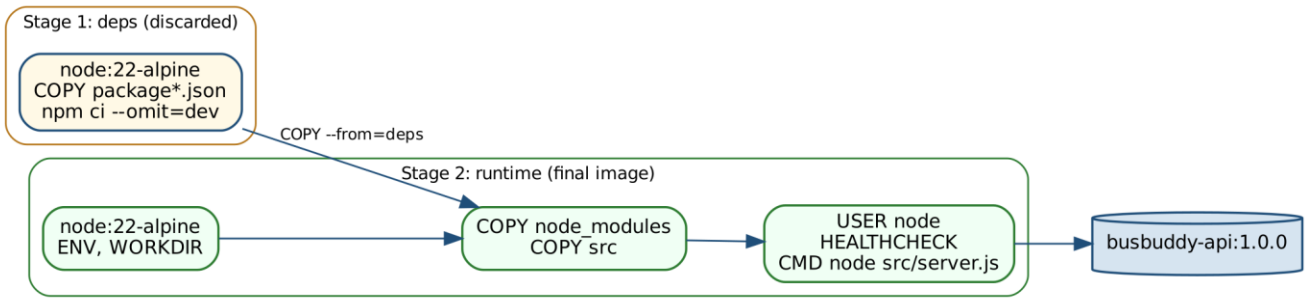


Figure 8: *Two-stage build of the BusBuddy API image.*

```
# syntax=docker/dockerfile:1

# ---- Stage 1: install production dependencies only ----
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
```

```
# ---- Stage 2: small runtime image ----
FROM node:22-alpine
ENV NODE_ENV=production \
    PORT=3000 \
    DB_FILE=/data/busbuddy.db
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
# the database folder must be writable by the non-root user
RUN mkdir /data && chown node:node /data
USER node
EXPOSE 3000
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/health || exit 1
CMD ["node", "src/server.js"]
```

8.1 Purpose of Each Instruction

Instruction	Purpose
# syntax=docker/dockerfile:1	Selects the current Dockerfile syntax (BuildKit).
FROM node:22-alpine AS deps	Starts stage 1 from a small Node.js 22 image; names it "deps".
WORKDIR /app	Sets the working folder inside the image.
COPY package.json package-lock.json ./	Copies only the dependency files first, so Docker caches the install layer until they change.
RUN npm ci --omit=dev	Installs exact locked versions, without development tools.
FROM node:22-alpine	Starts stage 2 (the final image) from a fresh base.
ENV NODE_ENV, PORT, DB_FILE	Default settings: production mode, port 3000, database path in the /data volume.
COPY --from=deps /app/node_modules	Takes only the installed packages from stage 1.
COPY src ./src	Adds the application code (tests and secrets are excluded by .dockerignore).
RUN mkdir /data && chown node:node /data	Creates the data folder and lets the non-root user write the database there.
USER node	Runs the app without root rights (security).
EXPOSE 3000	Documents the port the API listens on.
HEALTHCHECK … wget /health	Lets Docker test that the API is really answering.
CMD ["node","src/server.js"]	The command that starts the API when the container starts.

9. Docker Compose Design

docker-compose.yml describes the complete system. Figure 9 shows how the three services, the private network and the volume are connected.

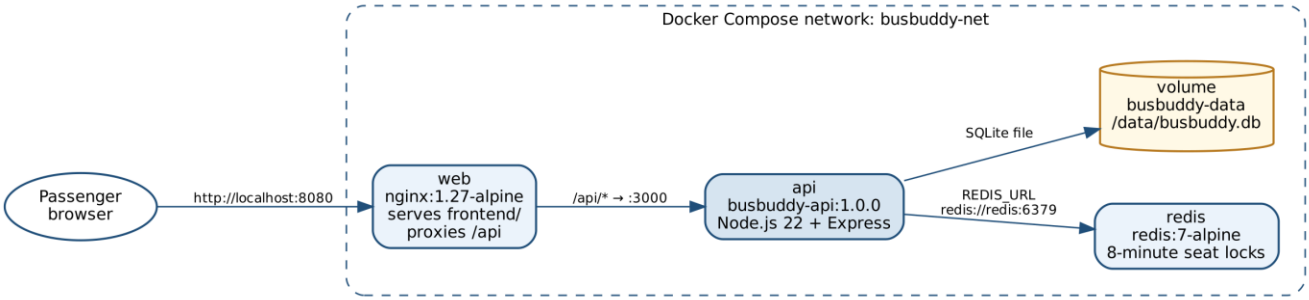


Figure 9: *BusBuddy container architecture started by Docker Compose.*

```
services:
  redis:
    image: redis:7-alpine
    command: ["redis-server", "--save", "", "--appendonly", "no"]   # locks are temporary
    networks: [busbuddy-net]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      timeout: 3s
      retries: 5
    restart: unless-stopped

  api:
    build: ./backend
    image: busbuddy-api:1.0.0
    environment:
      PORT: 3000
      REDIS_URL: redis://redis:6379
      DB_FILE: /data/busbuddy.db
    volumes:
      - busbuddy-data:/data
    depends_on:
      redis:
        condition: service_healthy
    networks: [busbuddy-net]
    restart: unless-stopped

  web:
    image: nginx:1.27-alpine
    ports:
      - "${WEB_PORT:-8080}:80"
    volumes:
      - ./frontend:/usr/share/nginx/html:ro
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - api
    networks: [busbuddy-net]
    restart: unless-stopped

networks:
  busbuddy-net:

volumes:
  busbuddy-data:
```

9.1 Services, Networking, Volumes, Variables and Dependencies

Item	Explanation
Services	**redis** (seat locks), **api** (business logic, built from ./backend) and **web** (Nginx, the only public service).
Networking	All services join the private network busbuddy-net. They find each other by service name (for example redis://redis:6379 and http://api:3000). Only web publishes a port to the host (8080 → 80); the API and Redis are not reachable from outside.
Volumes	Named volume busbuddy-data is mounted at /data in the api container, so the SQLite database survives restarts and rebuilds. The frontend and nginx files are mounted read-only (:ro) into web.
Environment variables	PORT, REDIS_URL and DB_FILE configure the API without changing code. WEB_PORT comes from the .env file (default 8080).
Dependencies	api waits until redis is healthy (depends_on with condition service_healthy, using redis-cli ping). web starts after api.
Reliability	restart: unless-stopped brings containers back after a crash or reboot. Redis persistence is switched off on purpose: locks are temporary and must expire.

10. Container Deployment and Testing

10.1 Deployment Steps

```
$ cp .env.example .env
$ docker compose up --build -d       # build the API image and start the 3 services
$ docker compose ps                  # check that api and redis are "healthy"
$ curl http://localhost:8080/health  # answer comes through Nginx from the API
$ docker compose logs -f api         # follow API logs
$ docker compose down                # stop; add -v to delete the data volume
```

10.2 Testing Procedure

Step	Test	Expected result	Evidence
1	Validate the Compose file	File is valid; services api, redis, web	Figure 10
2	Run automated tests (npm test)	6 tests pass (search, lock, book, refund slabs, boundary)	Figure 10
3	Call the API against a real Redis server	Lock 200 → second lock 409 → booking 201 → cancel with refund 675	Figure 11
4	Check lock expiry in Redis	TTL of a new lock is 480 seconds	Figure 11
5	Start the stack with Docker Compose	Three containers up; health check returns ok	Figure 12

10.3 Evidence

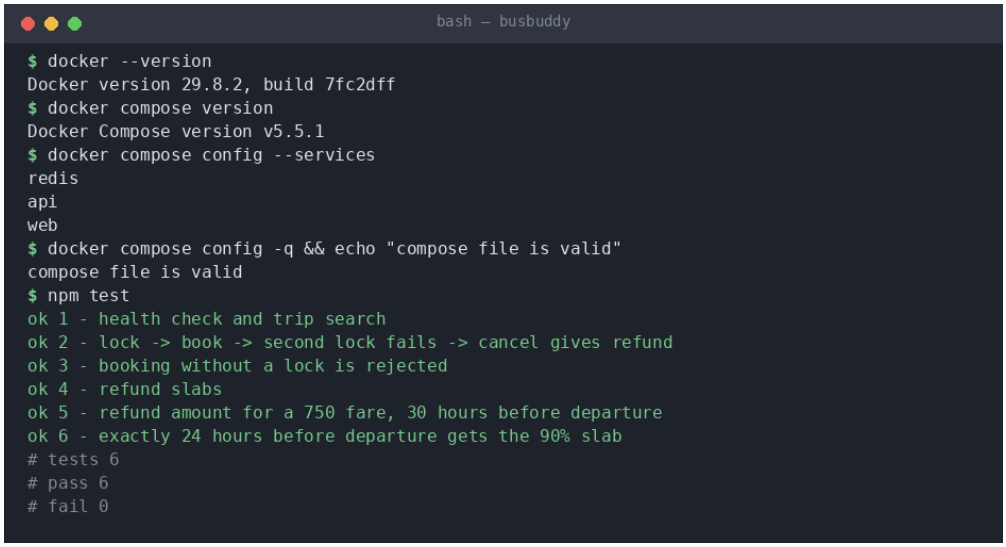


Figure 10: *Docker and Compose versions, validation of docker-compose.yml, and the 6 automated tests (all passed).*

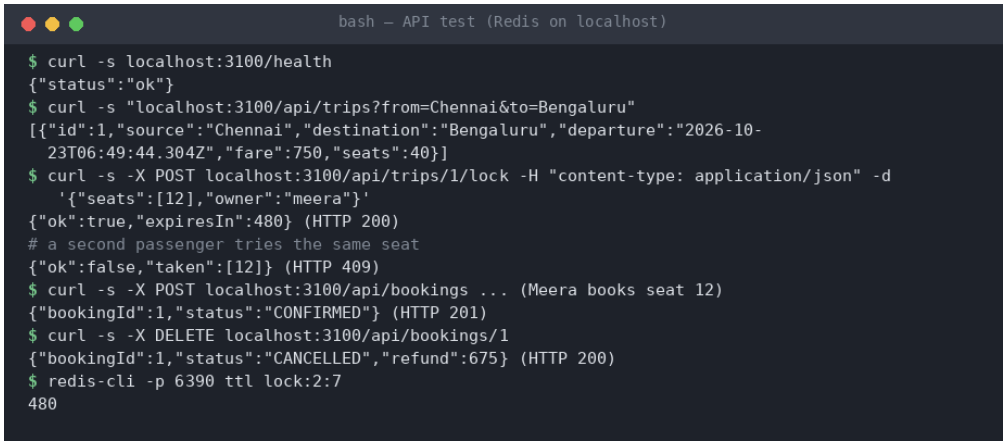


Figure 11: *API test against a real Redis server: the second passenger cannot take the locked seat; the cancelled booking returns ₹675 (90% of ₹750). The last command shows the seat lock expires in 480 seconds.*

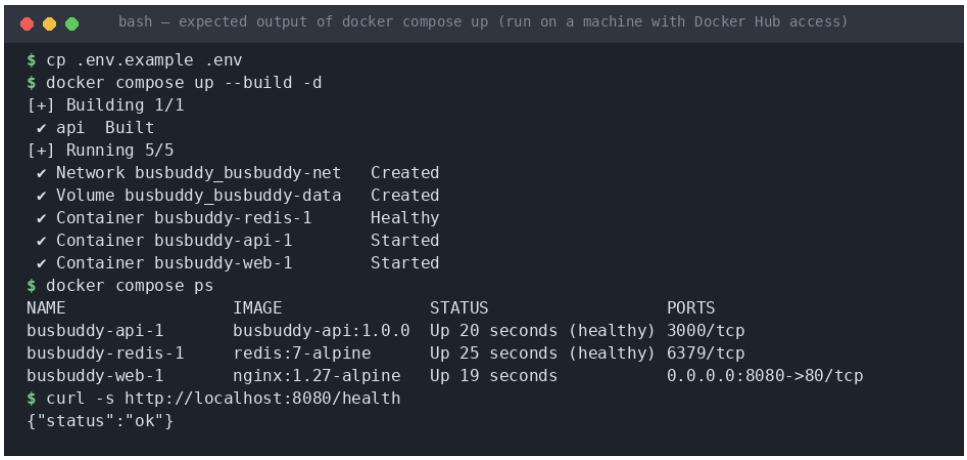


Figure 12: *Expected output of docker compose up for the BusBuddy stack.*

Evidence note
Figures 10–11 are real output from this project: the same API code and the same Redis commands that run inside the containers. Figure 12 shows the output expected from docker compose up on a computer that can download the base images (node, redis, nginx) from Docker Hub. Replace Figure 12 with a screenshot from your own run before submission.

11. Benefits of Git and Docker

Aspect	Git in BusBuddy	Docker in BusBuddy
Collaboration	Feature branches and pull requests let several people work without overwriting each other; the seat-lock and booking features were developed in parallel.	Every teammate runs the same stack with one command; no "install Node and Redis" instructions.
Version management	Tags v1.0.0 and v1.0.1 mark exact releases; any commit can be reverted.	Images are tagged (busbuddy-api:1.0.0), so an old version can be started again.
Portability	Whole project history travels in one repository.	Same containers run on a laptop, a college server or the cloud.
Deployment	CI runs tests on every push to main and develop.	docker compose up deploys; restart policy and health checks keep it running.
Scalability	Branching allows more developers to join safely.	More API containers can be added behind Nginx for peak booking days.
Maintainability	Readable commit messages and CHANGELOG explain why code changed (for example, the refund hotfix).	Small images, non-root user and a data volume make updates and clean-up simple.

12. Challenges and Improvements

12.1 Challenges Encountered

Challenge	How it was handled
Merge conflict in server.js (two features edited the same lines)	Read both versions, kept both routes, tested the file and recorded the decision in the merge commit (Section 5.1).
Partial seat locks: if one seat of a group was taken, earlier seats stayed locked	Fixed in commit "fix(locks): stop at first taken seat and release partial locks" so locking is all-or-nothing.
Refund boundary bug: exactly 24 hours before departure gave 50% instead of 90%	Found after release 1.0.0; fixed on a hotfix branch with a new test, tagged v1.0.1 and merged to main and develop.
Non-root container could not write the SQLite file	Created /data in the Dockerfile and gave it to the node user; the folder is a named volume.

Challenge	How it was handled
API started before Redis was ready; tests should not need a real Redis	Added a Redis health check with depends_on (service_healthy) in Compose, and an in-memory fake Redis so tests run anywhere and in CI.

12.2 Possible Improvements and Future Enhancements

Area	Improvement
Git	Protect main and develop with required reviews and passing CI; add pre-commit hooks and signed commits; automate version numbers and the changelog from commit messages.
Docker	Push images to a registry and scan them for vulnerabilities; add a production Compose file with HTTPS; later deploy continuously and move to Kubernetes.
Docker	Replace SQLite with PostgreSQL in its own container for many concurrent bookings; run several API containers behind Nginx.
Application	Add payment gateway, e-ticket QR code, waitlist and live tracking from the SRS as new feature branches.

> **Summary**
> BusBuddy was developed in a Git repository of **34 commits** with feature, release and hotfix branches, one resolved merge conflict and two tags. It is packaged with a **multi-stage Dockerfile** and a **three-service Docker Compose** stack. Automated tests and live API tests with Redis confirm that a seat cannot be sold twice and that refunds follow the slab.

References

[1] Git Documentation – https://git-scm.com/doc

[2] Pro Git (S. Chacon, B. Straub) – https://git-scm.com/book

[3] V. Driessen, “A successful Git branching model”, 2010 – https://nvie.com/posts/a-successful-git-branching-model/

[4] Conventional Commits 1.0.0 – https://www.conventionalcommits.org

[5] Docker Documentation: Dockerfile and Compose file reference – https://docs.docker.com/reference/

[6] Redis Documentation: SET with NX and EX options – https://redis.io/docs/latest/commands/set/